



Faculty of Engineering & Technology

Electrical & Computer Engineering Department

REAL-TIME SYSTEM AND INTERFACING

TECHNIQUES LABORATORY

ENCS5140

Report #1

Arduino Experiments

Prepared by:

Yasmeen Kamal 1201146

Lana Batnaj 1200308

Aya R Dahbour 1201738

Instructor: Dr. Hanna Bullata

Teaching Assistant : Eng. Rania Shahwan

Section : 1

Date: 4/11/2024

Abstract

This report provides an overview of three experiments designed to introduce key functionalities of the Arduino module. The first experiment covers Arduino's basic programming features, demonstrating how digital and analog pins can read sensor data, with a focus on light measurement using an LDR and controlling an LED based on sensor values. It also addresses handling hardware and software disruptions. The second experiment introduces the fundamentals of serial communication, highlighting the use of Arduino's UART interface for real-world applications. The final experiment examines the I2C protocol, exploring device addressing and connecting multiple devices to Arduino via I2C.

Table of Contents

Abstract.....	1
List Of Figures.....	5
1 Theory.....	6
1.1 Introduction to Arduino	6
1.1.1 Arduino:.....	6
1.1.2 Arduino Uno	6
1.1.3 Photocells	7
1.1.4 Interrupts.....	7
1.2 Serial communication with Arduino	8
1.2.1 Definition of UART	8
1.2.2 How it works with Arduino	9
1.2.3 Timing and Synchronization	10
1.2.4 Baud Rate	10
1.2.5 UART Messages	11
1.2.6 Start Bit.....	11
1.2.7 Data Bits.....	11
1.2.8 Parity	12
1.2.9 Stop Bit.....	12
1.3 I2C in Arduino.....	13
1.3.1 I2C protocol overview.....	13
1.3.2: I2C data transfer and timing	14
1.3.3: LM75B temperature sensor	16
2 Procedure and discussion	17
2.1 Introduction to Arduino	17
2.1.1 Photoresistor.....	17
2.1.2 Hardware interrupts.....	19
2.2 The serial communication with Arduino.....	21
2.2.1 Basic communication between Arduino & PC	21
2.2.2: Basic communication between 2 Arduinos	22
2.2.3 Push Button & LED using 2 Arduinos	23

2.2.4 Visualization of serial communication using Serial Plotter	25
2.2.5 TODO: LED Brightness control Via Push Buttons and Using UART	27
2.3 I2C in Arduino.....	28
2.3.1 Writing the LCD to Arduino and I2C scanning	28
2.3.2: Display static text on the LCD	29
2.3.3: I2C communication between 2 Arduinos.....	29
2.3.4: I2C communication between 2 Arduinos and LCD	30
2.3.5: I2C communication between Arduino and Temperature sensor (LM75B)	31
2.3.6 ToDo	32
3. Conclusion	33
4. References	34
Appendix	35
Introduction to Arduino	35
Photoresistor	35
Software Interrupts.....	36
Hardware Interrupts	37
UART	38
Appendix 2.1:.....	38
Appendix 2.2:.....	38
Appendix 2.3:.....	39
Appendix 2.4:.....	40
Appendix 2.5:.....	40
I2C in Arduino.....	43
Part 1:.....	43
Part 2	43
Part 3	44
- Master	44
- Slave	44
Part 4	45
- Master	45
- Slave	46

Part 5 46

ToDo 47

- **Master** 47

- **Slave** 48

List Of Figures

Figure 1:arduino uno.....	6
Figure 2:Photocells.....	7
Figure 3:Resistance vs. illumination	7
Figure 4: Direct Communication between two Controllers by UART [7]	9
Figure 5: Two Arduino connecting by UART.....	9
Figure 6: Serial Communication [8].....	10
Figure 7: UART Frame Format [8].....	11
Figure 8: Start Bit [8]	11
Figure 9: Data Bits [8].....	12
Figure 10: Parity Bit [8]	12
Figure 11: Stop Bit [8]	13
Figure 12: I2C protocol	13
Figure 13: SDA Data Change.....	14
Figure 14: I2C Start and stop bits transfer	14
Figure 15: Example of Single Byte Data Transfer.....	15
Figure 16: LM75B temperature sensor	16
Figure 17:Simple demonstration of Photocells sensors.....	17
Figure 18:Serial Monitor	18
Figure 19:Hardware interrupt connection.	19
Figure 20 :Software Interrupt	20
Figure 21:Part 1	21
Figure 22: Result of the first part	21
Figure 23: Part 2	22
Figure 24: Part 2 connection	22
Figure 25: Part 2 Result	23
Figure 26: Part 3 circuit	24
Figure 27: Part 3 connection	24
Figure 28: Part 3 Results.....	25
Figure 29: Part 4 Circuit.....	25
Figure 30: Part 4 Connection.....	26
Figure 31: Part 4 Result	26
Figure 32:: Todo Connection	27
Figure 33:Find the address connection.....	28
Figure 34: Serial Monitor output for address	28
Figure 35: I2C communication between 2 Arduinos connection	29
Figure 36: I2C communication between 2 Arduinos master receiving	30
Figure 37: I2C communication between 2 Arduinos and LCD connection.....	30
Figure 38: I2C communication between 2 Arduinos and LCD serial monitor & LCD output	31
Figure 39: I2C communication between Arduino and Temperature sensor connection and serial monitor output	31
Figure 40: ToDo connection	32

1. Theory

1.1 Introduction to Arduino

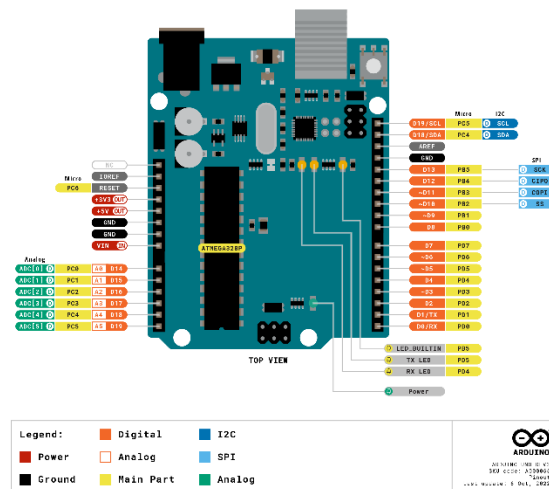
1.1.1 Arduino:

Arduino is an open-source platform that makes it easy to create electronic projects. It includes a physical circuit board, or microcontroller, and software called the IDE (Integrated Development Environment) that runs on a computer to write and upload code to the board.

Arduino has become especially popular among beginners in electronics because of its simplicity. Unlike many older boards that required extra hardware (called a programmer) to load code, Arduino allows you to upload code directly with a USB cable. The IDE also uses a simplified version of C++, making programming easier to learn. Additionally, Arduino boards come in a standardized layout that organizes the microcontroller's functions into an easy-to-use format, making it more accessible for everyone. [1]

1.1.2 Arduino Uno

One type of Arduinos boards is Arduino-uno which has a lot of characteristics for his low cost, flexible, and easy-to-use capabilities, as microcontroller board that can be integrated into a variety of hardware and electronic projects. This board can be interfaced with other versions of Arduino boards, Arduino shields, and other microcontrollers from other platforms and can control relays, LEDs, servos, and motors as an output. Arduino UNO features AVR microcontroller Atmega328, 6 analogue input pins, and 14 digital I/O pins out of which 6 are used as PWM or - Pulse Width Modulation – output . [2]



1.1.3 Photocells

LDRs or photoresistors, are sensor devices that are sensitive to light which are most often used to indicate the presence or absence of light, or to measure the light intensity. It works by making their resistance very high in the dark -sometimes up to $1\text{ M}\Omega$ -, and when it's exposed to light, the resistance drops dramatically, even down to a few ohms, depending on the light intensity. This sensor is a nonlinear device that has a sensitivity that changes with the wavelength of the light applied. They are used in many applications, but this light sensing function is often performed by other sensor devices . [3]



Figure 2:Photocells.

The sensor is made of semiconductor materials that changes its resistance when exposed to light such as cadmium sulfide -CdS-, that happens when the light strikes the semiconductor material, it excites electrons in the material, allowing them to move more freely and by that reducing the overall resistance of the material.

As the resistance can as be high as $10\text{M}\Omega$ and decreases as light increases. [4]

The following figure shows the relationship between them.

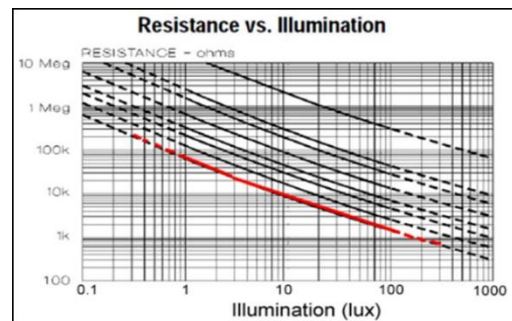


Figure 3:Resistance vs. illumination

1.1.4 Interrupts

The interrupt is a signal which is used to tell the Arduino to stop what its doing and move to deal with another process that has more priority, the process of interrupt happens as follows, first the Arduino get the interrupt signal then the Arduino stops the current process and store

its current state to save the state he was at and then performs a function which is called the ISR of the interrupt service routine, there are two main types of hardware and a software interrupts which we will discuss next.

A. Hardware interrupts:

The hardware interrupts is a signal the Arduino got from other devices which is external, the Arduino has 2 pins dedicated for the external interrupts which is pins 2, and 3 -int 0 and int 1- [6] . These pins are usually connected to an external device, and then an action or an interrupt happens, the pins receive a signal and then ISR is performed.

To identify an interrupt and execute the ISR (Interrupt Service Routine), the interrupt must be attached with the following line: “**attachInterrupt(interrupt, function, mode);**” with all of this: [5]

- **interrupt:** Specifies the interrupt number (or pin).
- **function:** The ISR to be called when the interrupt occurs.
- **mode:** Defines the pin state that triggers the interrupt (e.g., RISING, FALLING, CHANGE, LOW, HIGH).

B. Software interrupts:

This type of interrupts is caused by the software -the code itself-, such as special instructions and exceptions, or as in this experiment a timer overflow, ATmega168/328 microcontroller includes a 16-bit hardware timer, Timer1, which was chosen for this experiment due to its ability to trigger interrupts under specific conditions. It also offers multiple configuration options, such as PWM (Pulse Width Modulation), CTC (Clear Timer on Compare Match), and Normal mode [6].

1.2 Serial communication with Arduino

1.2.1 Definition of UART

UART, or universal asynchronous receiver transmitter, is a frequently employed protocol for device-to-device communication.

When properly set up, UART may be used with various serial protocols to receive and send serial data. Serial communication transfers data bit by bit across just one line or wire. Two wires are used for two-way communication to exchange serial data successfully. Based on the application and system needs, serial communication involves fewer components and cables, resulting in lower implementation costs [7].

UART is primarily a hardware communication mechanism between embedded devices, microcontrollers, and computers.

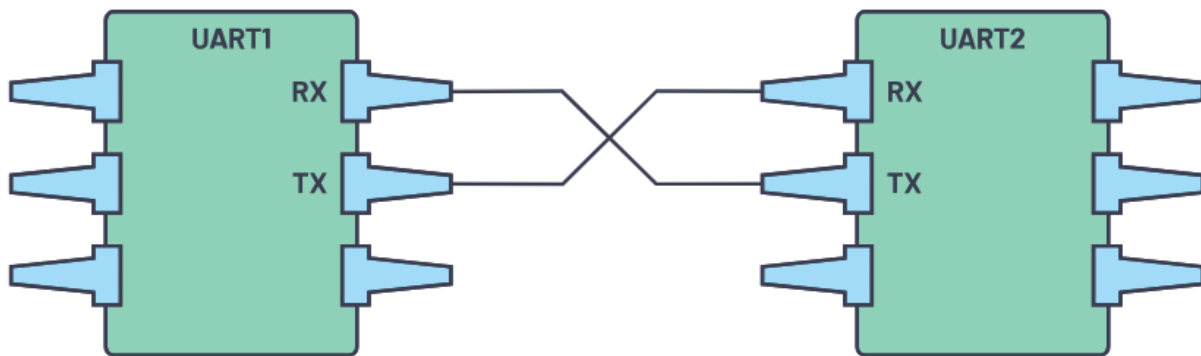


Figure 4: Direct Communication between two Controllers by UART [7]

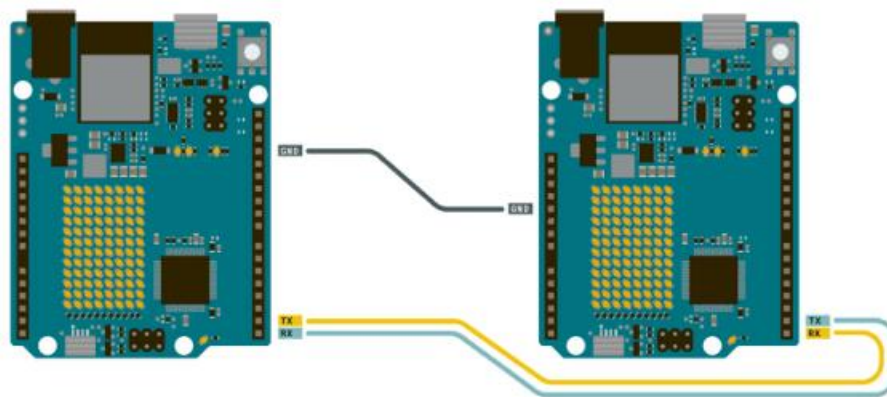


Figure 5: Two Arduino connecting by UART

Despite its widespread usage as a hardware communication protocol, it is not always properly optimized. When employing the UART module inside the microcontroller, proper implementation of the frame protocol is sometimes overlooked.

Every UART device has two electrical signals that are named:

- Transmitter (Tx)
- Receiver (RX)

The primary function of a transmitter and receiver line in every device is to transmit and receive serial data intended for serial communication.

1.2.2 How it works with Arduino

UART transmits data as a series of bits, which include a start bit, data bits, a potential parity bit, and stop bit(s). Unlike parallel communication, which transmits many bits concurrently, UART

transfers data serially, one bit at a time. As the name implies, the protocol runs asynchronously, which means it does not rely on a common clock signal. Instead, it employs specified baud rates to time data bits [8].

Serial allows the Arduino board to communicate with a computer or other devices. All Arduino boards contain a serial port, sometimes called a UART or USART. The device uses digital pins 0 (RX) and 1 (TX) for communication and connects to computers via USB. Using these routines prevents you from using pins 0 and 1 for digital input or output.

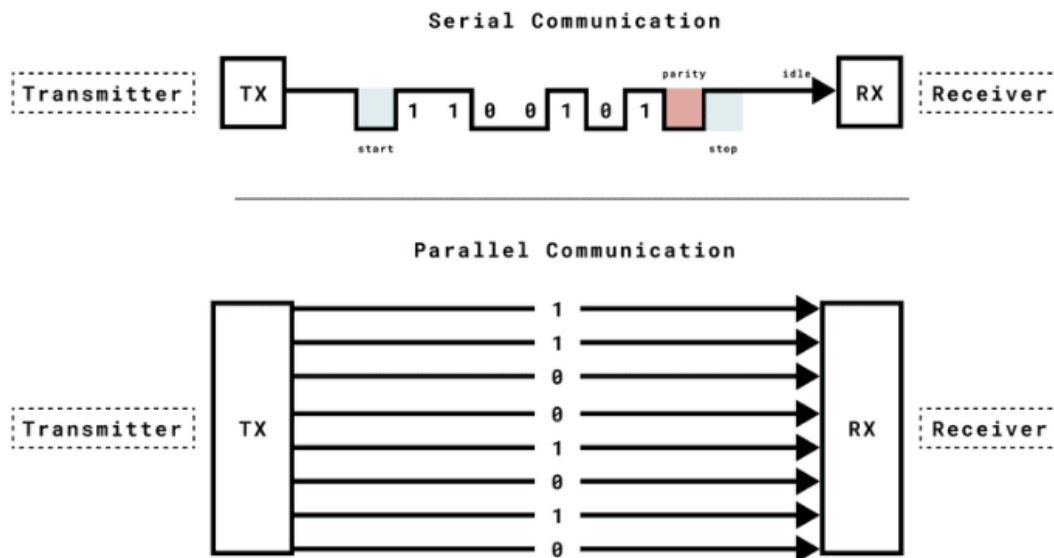


Figure 6: Serial Communication [8]

As observed in the image above, when utilizing parallel transmission, an 8-bit message would require eight wires, but serial communication requires just one cable for transmitting and one for receiving.

1.2.3 Timing and Synchronization

Timing and synchronization are critical components of UART communication. Unlike synchronous serial communication protocols like SPI and I2C, UART runs asynchronously, which means it does not use a common clock signal to coordinate data transfer. Instead, it employs specified baud rates to time data bits [8].

1.2.4 Baud Rate

The baud rate is an important component in UART communication. It specifies the rate at which data is transferred across the communication channel. The baud rate (bps) represents the number

of bits sent in one second. To communicate successfully using UART, both the sending and receiving devices must agree on the same baud rate [8].

1.2.5 UART Messages

Each data frame in the UART connection is encoded with start and stop bits. These bits play an important function in defining data transmission boundaries and guaranteeing synchronization between sender and recipient [8].

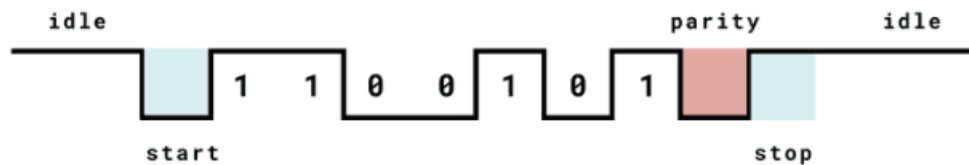


Figure 7: UART Frame Format [8]

1.2.6 Start Bit

A single start bit is sent at the start of each UART frame. The primary function of the start bit is to signal the beginning of data transfer and prepare the receiving device for data reception. The start bit is always logic low (0) when communicating over UART. This indicates that the start bit is communicated at a voltage level lower than the logic high threshold, usually at the receiver's end. When the receiver detects a start bit, it recognizes that a new data frame has begun and prepares to accept the incoming bits [8].

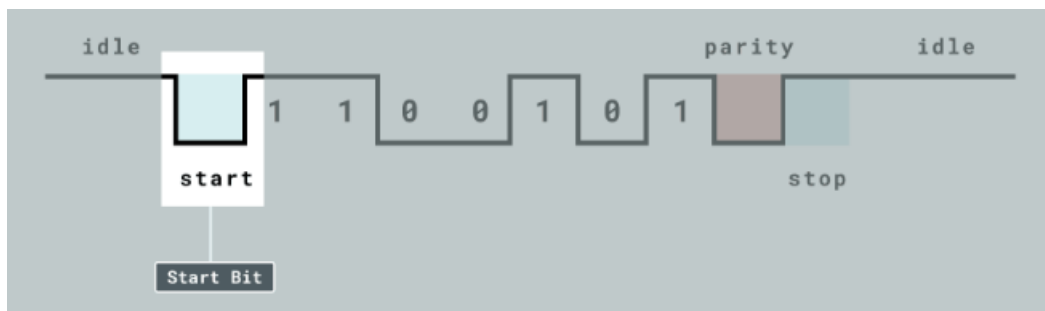


Figure 8: Start Bit [8]

1.2.7 Data Bits

Data bits are a vital element of UART communication since they contain the true data that needs to be conveyed. The amount of data bits in a UART frame might vary, but the most common and extensively used arrangement is 8 bits. Depending on the application, UART can handle a variety of character sizes, including 7-bit and 6-bit versions [8].

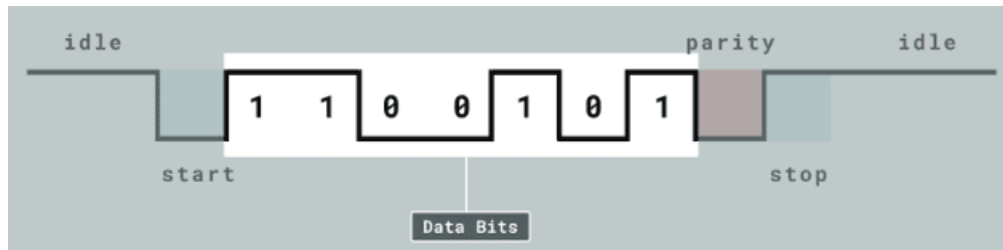


Figure 9: Data Bits [8]

In UART communication, character size is determined by the amount of data bits in a frame. It is critical to select a suitable character size to meet the needs of the data being conveyed. Here are some typical character size configurations:

8-Bit: The most common character size in UART transmission. It enables the transfer of a single byte of data, which can include a variety of values such as ASCII characters, numerical values, and so on.

7-Bit: When data size needs to be reduced, a 7-bit character size is used. It is appropriate for applications that need minimal data overhead and can represent 128 distinct values.

6-Bit: For even more concise data representation, utilize a 6-bit character size. This setup supports 64 distinct settings [8].

1.2.8 Parity

Alongside the data bits, UART transmission may contain a parity bit within the data frame. Parity is a method for detecting faults in data transfer. Parity can be adjusted to "odd" or "even," ensuring that the total number of bits set to logic "1" in a character is either even or odd, depending on the parity type selected. The existence of a parity bit enables the receiver to verify the integrity of the received data. If the number of "1" bits does not correspond to the intended parity, an error is identified [8].

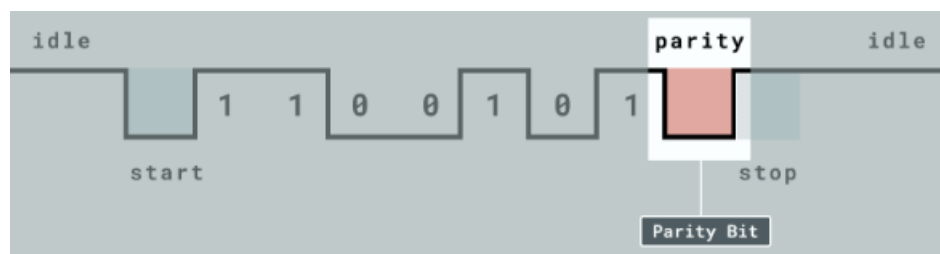


Figure 10: Parity Bit [8]

1.2.9 Stop Bit

The stop bit(s) signify the end of the data byte and the finish of data transmission. The most typical arrangement is to utilize one one-stop bit; however, two-stop bits can be used for enhanced dependability.

The polarity of the stop bit(s) might differ, with some systems utilizing a high stop bit and others using a low stop bit, depending on the UART setup [8].

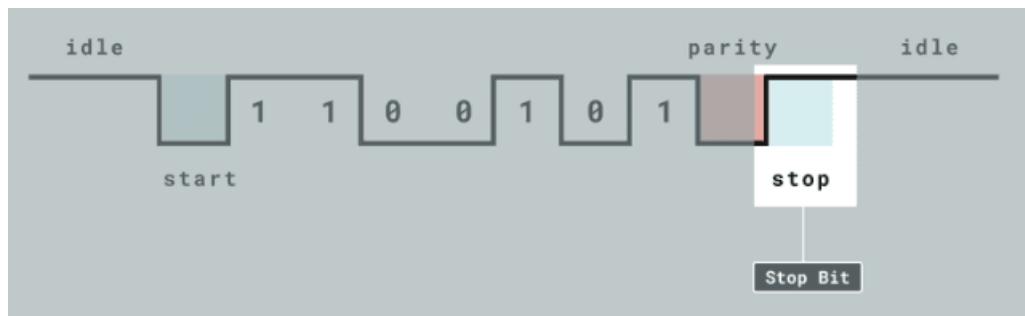


Figure 11: Stop Bit [8]

1.3 I2C in Arduino

1.3.1 I2C protocol overview

I2C, or Inter-Integrated Circuit, is a two-wire half-duplex serial communication interface developed by Philips Semiconductors (now NXP Semiconductors) in the 1980s. It is used to connect low-speed devices, such as sensors and microcontrollers, with each other or with higher-level devices such as controllers, memory chips, or other integrated circuits. The I2C protocol was designed to reduce the number of wires required to connect multiple devices and to allow simple and efficient communication between them.

Like UART communication, I2C only uses two wires to transmit data between devices:

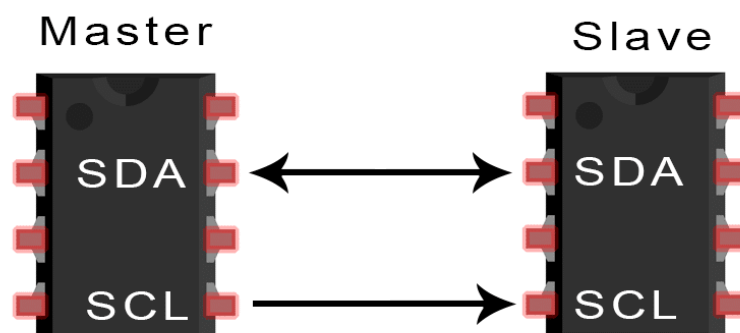


Figure 12: I2C protocol

SDA, the data line for master and slave communication, transfers bits serially. Along this single wire, information is conveyed from device to device. [9]

SCL provides the clock, synchronizing the output and sampling of bits between devices on the bus. As director of timing, the clock signal remains under the master's control.

I2C uses a serial protocol, sending data one bit at a time along the SDA line. Like SPI, it works synchronously. Both systems rely on a jointly held clock signal to pace transmission and reception of bits between the master and any slaves addressed. Precise timing is enforced through the clock, with the master laying down the beat.

1.3.2: I2C data transfer and timing

1.3.2.1 Bit Transfer

For each clock pulse, a single data bit is transferred on the SDA line, and the data must remain stable when the SCL line is high to avoid any misinterpretation of data. Any changes on SDA when SCL is high are considered control signals like START or STOP conditions. [10]

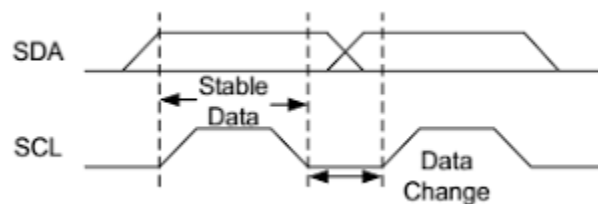


Figure 13: SDA Data Change

1.3.2.2 Start and stop conditions

All I2C transactions begin with a START (S) and are terminated by a STOP (P).

- START condition: When a HIGH to LOW transition occurs on the SDA line while SCL is HIGH.
- STOP condition: When a LOW to HIGH transition occurs on the SDA line while SCL is HIGH. [11]

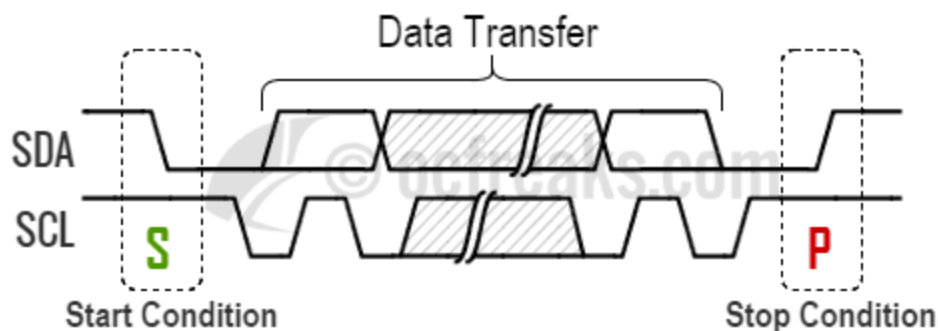


Figure 14: I2C Start and stop bits transfer

1.3.2.3 I2C Data Transfer

In the I2C data transmission protocol, the information is transmitted across the bus in packets of 8 bits normally referred to as bytes. After each of the bytes transmitted, an acknowledgement bit is returned to the checking of whether the slave device has successfully received data. All the data bits and the ACK bit required for the master device are generated. If the slave does not send an ACK, it means that either it is not ready or it has no further data to be transferred. Thus, the master must send a Stop condition to allow the bus release, or a Repeated Start condition in case it has further communications to make without releasing the bus. This arrangement ensures orderly data transfer and flow control at the I2C bus. [12]

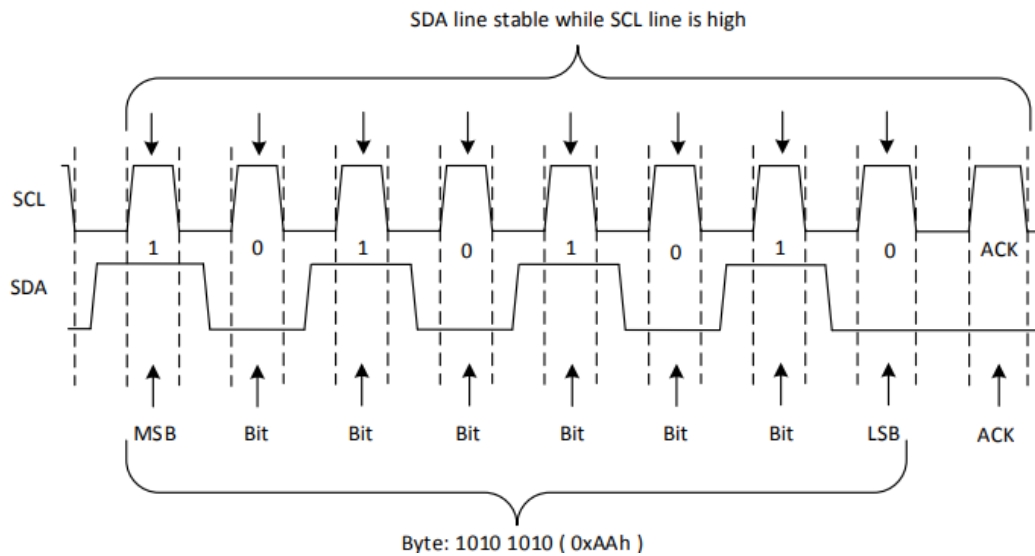


Figure 15: Example of Single Byte Data Transfer

1.3.2.4 I2C Synchronization

When there are multiple masters connected in an I2C bus, synchronization refers to the coordination of the clocks of all masters such that the data transfer proceeds properly. Though every master generates its own SCL signal, all the masters are observing the SCL line. If one of the masters pulls the clock line low, then it remains low until the other masters also release it. This is exactly how the masters can "stretch" the clock and share the lowest clock frequency. Once the clock is released, the SCL stays high until the first master pulls it low again.

1.3.2.5 Arbitration

Even though multiple masters may be present on the I2C bus the arbitration is handled in such a way that there is no corruption of data on bus in case when more than 2 masters try to transmit data at the same time. Since the transmission, synchronization and arbitration is done using only

2 wires on the bus, the communication protocol might be a bit uneasy to understand for beginners ... but it's actually easy to understand.

1.3.2.6 Communication with 7-BIT I2C Addresses

In 7-bit I2C addressing, communication starts with the master issuing a START condition, followed by the 7-bit address of the slave and a read/write (R/W) bit. The slave acknowledges by pulling the SDA line low, allowing data transfer to proceed in 8-bit bytes with an ACK bit after each. The master ends communication with a STOP or repeated START condition. For switching from write to read, a repeated START is used with the read bit set. This flexible approach enables communication with multiple devices on the same bus.

1.3.3: LM75B temperature sensor

LM75 Temperature Sensor module is a delta-sigma analog to digital converter. The cost-effective device consumes less power and is highly precise. An over-temperature protection circuit is integrated to sink the current that shut down in the module in case of over the limit temperature. LM75 communicates the data using the I2C interface. To connect multiple hosts to the same bus, three address pins are available for the purpose.



Figure 16: LM75B temperature sensor

2. Procedure and discussion

2.1 Introduction to Arduino

2.1.1 Photoresistor

In this section of the test, we will see the impact of adjusting the photoresistor's value on the LED's brightness, using the Arduino for control and monitoring. The diagram presented in the figure was built specifically for this objective.

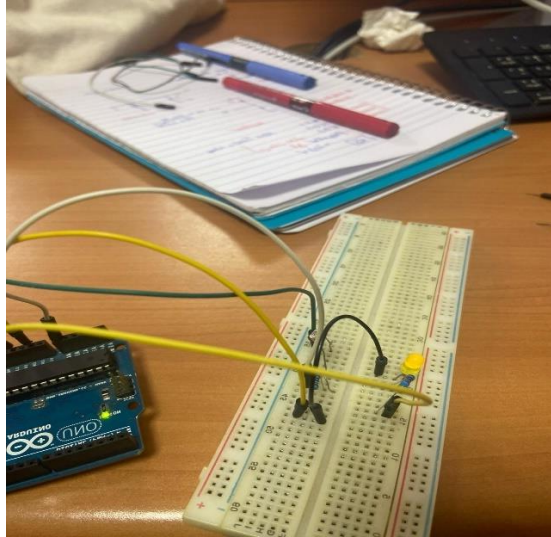


Figure 17: Simple demonstration of Photocells sensors.

The photoresistor is attached to a 10 k Ω resistor and connected to the analog port A1 on the Arduino. The LED is connected to pin 11 on the Arduino, and the photoresistor is connected to the other end. It's essential to understand that the photoresistor measures analog values, whereas the pin sends out a digital signal.

After the connections are finished, the LED's light intensity is regulated according to the photoresistor's data, utilizing the Arduino's PWM pins. In the program (found in the appendix), the serial monitor shows the readings from the photoresistor as well as the light conditions determined by specific threshold values dark under 220, light under 620, and bright above 620.

Once the analog reading from the photoresistor is translated to a range of 255 to 1023, the LED brightness is adjusted based on these readings. This configuration allows for accurate adjustments of the LED's brightness based on variations in light detected by the photoresistor. The outcomes of changing the levels of light detected by the photoresistor are as stated below:

```
22 // LED gets brighter the darker it is at Dis
23 // that means we have to invert the readin
24 photocellReading = 1023 - photocellReading;
25 //now we have to map 0-1023 to 0-255 since t
26 LEDbrightness = map(potocellReading, 0, 102
27 analogWrite(LEDpin, LEDbrightness);
28
29
30 delay(1000);
31 }
Output Serial Monitor x
Message Editor and message to Arduino Uno on COM190
Analog reading = 308 - Light
Analog reading = 307 - Light
Analog reading = 302 - Light
Analog reading = 286 - Light
Analog reading = 276 - Light
Analog reading = 284 - Light
Analog reading = 161 - Dark
Analog reading = 90 - Dark
Analog reading = 97 - Dark
Analog reading = 96 - Dark
Analog reading = 609 - Light
Analog reading = 609 - Light
Analog reading = 652 - Bright
Analog reading = 654 - Bright
Analog reading = 654 - Bright
```

Figure 18:Serial Monitor

For the different lighting scenarios displayed in the figure:

1. **Dark Case**

In Dark Mode, the LED turns on when the area around the photocell becomes dark. This is achieved by getting an analog reading from the photocell, inverting it, and mapping it to a range of [0-255] to fit within the analogWrite() function. When the photocell reads less than 300, the LED lights up with an inverted brightness value, controlled by PWM. The code for this setup can be found in the Appendix. As shown in the figure, the reading is between 90-96, which is below the threshold of 220, so the serial monitor displays “Dark.”

2. **Light Case**

In Light Mode, the photocell reading falls between the threshold values, indicating a moderate light level. As seen in the figure, the reading is between 308-503, which is below 620 but above 220. The serial monitor outputs “Light” along with the reading.

3. **Bright Case**

In Bright Mode, the photocell detects a high light level. When the reading exceeds 620, the serial monitor outputs “Bright” along with the current reading, as shown in the figure.

These three scenarios demonstrate how the LED brightness responds to varying light levels, with readings and states accurately reflected on the serial monitor.

2.1.2 Hardware interrupts

In this section of the study, we will examine hardware interrupt theory by using an external push button and an Arduino to manage the LED's status. Initially, we put together the circuit depicted in the below figure. The push button is linked to pin 2 of the Arduino, whereas the LED is connected to the built-in LED pin. This illustration shows the utilization of a hardware interrupt triggered by the push button to manage the LED.

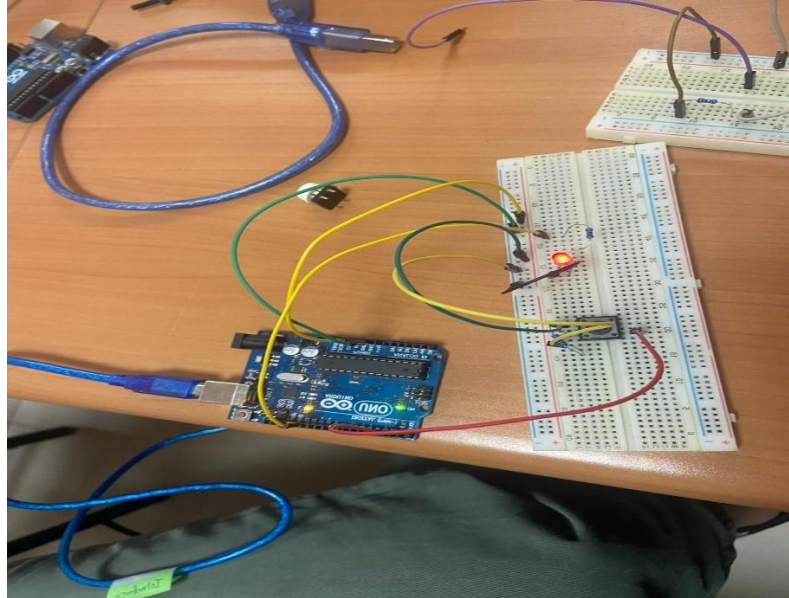


Figure 19: Hardware interrupt connection.

Pressing the button results in an interrupt being triggered, leading to a change in the state of the LED. The LED is turned off by setting the initial state to LOW. The code's `setup()` function configures an interrupt for pin 2 and connects the interrupt service routine to the `blink()` function. When pin 2 detects an interrupt, the `blink()` function is triggered and it changes the state of the LED.

Our observation showed that the LED is turned on when the button is pressed, and remains off when the button is not pressed, its default state. I possess a video displaying the act of pressing the button as opposed to a photograph.

<https://drive.google.com/file/d/1yvkb58EV9YkLJLIMrdLt2hKFnjOiS5dT/view?usp=sharing>

2.1.3 Software interrupts

In this part of the experiment, we will study the concept of software interrupts using the Arduino's Timer 1 to create an overflow interrupt. Unlike hardware interrupts, software interrupts are generated by the internal timer. Although the Arduino's built-in LED is typically

used for such experiments, due to a malfunction in the board's integrated LED, we used an external LED instead, as seen in the provided circuit image.

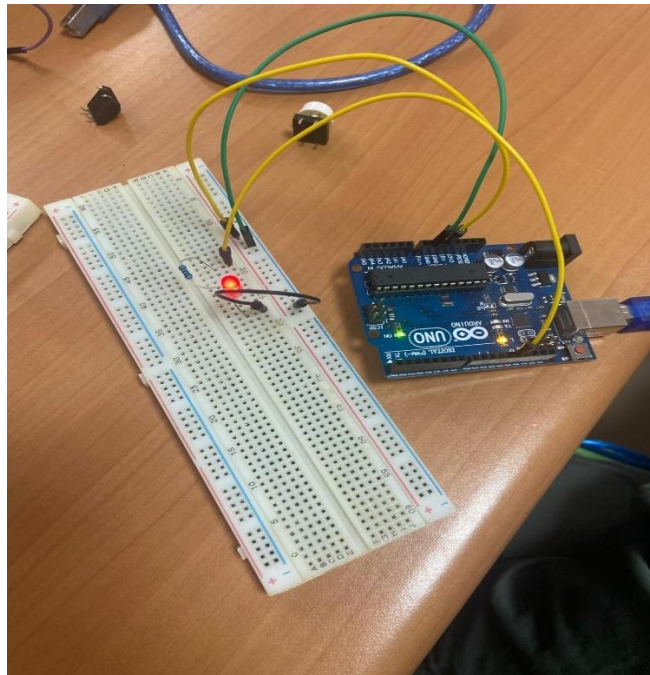


Figure 20 :Software Interrupt

Here's a summary of the setup and code behavior:

The Arduino is powered and connected to the external LED on the breadboard.

Timer 1, a 16-bit timer, is configured to create a software interrupt every 0.5 seconds.

The registers TCCR1A and TCCR1B are reset, and a prescaler of 256 is applied to reduce the Arduino's 16MHz clock frequency.

A preload value of 34286 is calculated and stored in the TCNT1 register to achieve a square wave frequency of 2Hz.

When the timer overflows, an interrupt service routine toggles the external LED. This process flips the LED on and off, simulating the effect of a blinking LED with a period of 0.5 seconds.

In this experiment, the key adjustment was replacing the built-in LED with an external one due to the Arduino's hardware issue. The external LED behaves as expected, controlled by the software interrupt triggered by the timer overflow.

https://drive.google.com/file/d/12MOH4Gj7M_2y_pXcK2gPxESxeoQdaktY/view?usp=sharing

2.2 The serial communication with Arduino

2.2.1 Basic communication between Arduino & PC

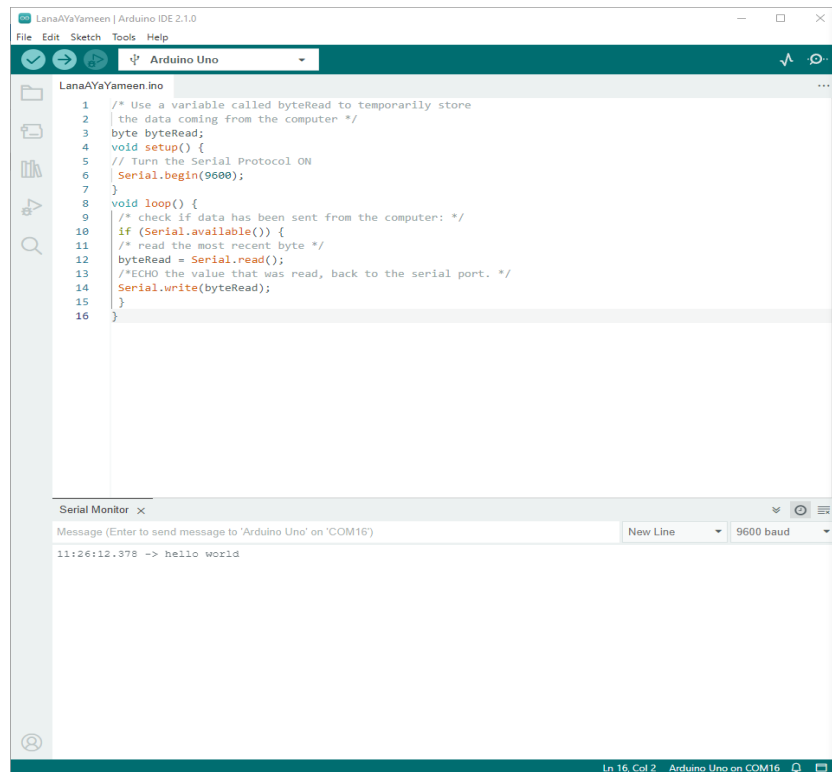
This section aims to familiarize users with the connections and setup by sending a basic message ["Hello World!"] between the Arduino and PC.

This code receives input serial data from the computer, saves it in `byteRead`, and then right away returns it to the serial port. It regularly checks for new information in the loop, which allows serial communication.



Figure 21:Part 1

NOTE: code in Appendix 2.1



```
1  /* Use a variable called byteRead to temporarily store
2  | the data coming from the computer */
3  byte byteRead;
4  void setup() {
5  | // Turn the Serial Protocol ON
6  | Serial.begin(9600);
7  }
8  void loop() {
9  | /* check if data has been sent from the computer: */
10 | if (Serial.available()) {
11 |   /* read the most recent byte */
12 |   byteRead = Serial.read();
13 |   /*ECHO the value that was read, back to the serial port. */
14 |   Serial.write(byteRead);
15 | }
16 }
```

Serial Monitor x

Message (Enter to send message to 'Arduino Uno' on 'COM16') New Line 9600 baud

11:26:12.375 -> hello world

Ln 16, Col 2 Arduino Uno on COM16

Figure 22: Result of the first part

2.2.2: Basic communication between 2 Arduinos

To create a simple interaction between two Arduinos, the message "Hello World!" was sent. The TX pin of the first Arduino connects to the RX pin of the second, and vice versa (TX1 ↔ RX2, RX1 ↔ TX2). The two Arduinos additionally have a common ground.

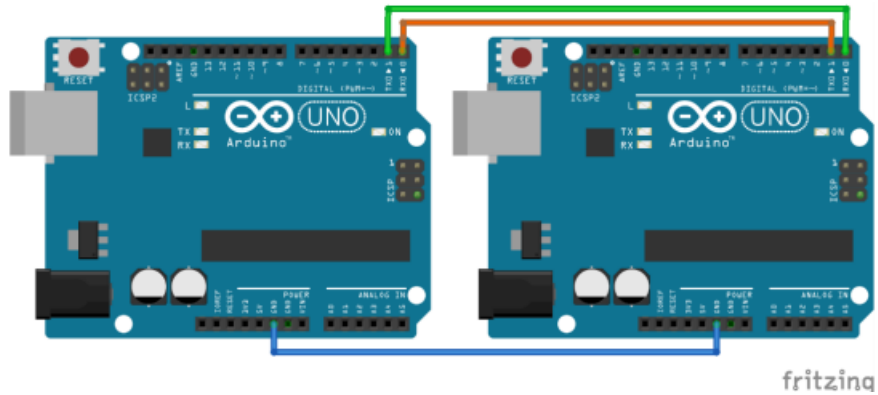


Figure 23: Part 2



Figure 24: Part 2 connection

NOTE: code in Appendix 2.2

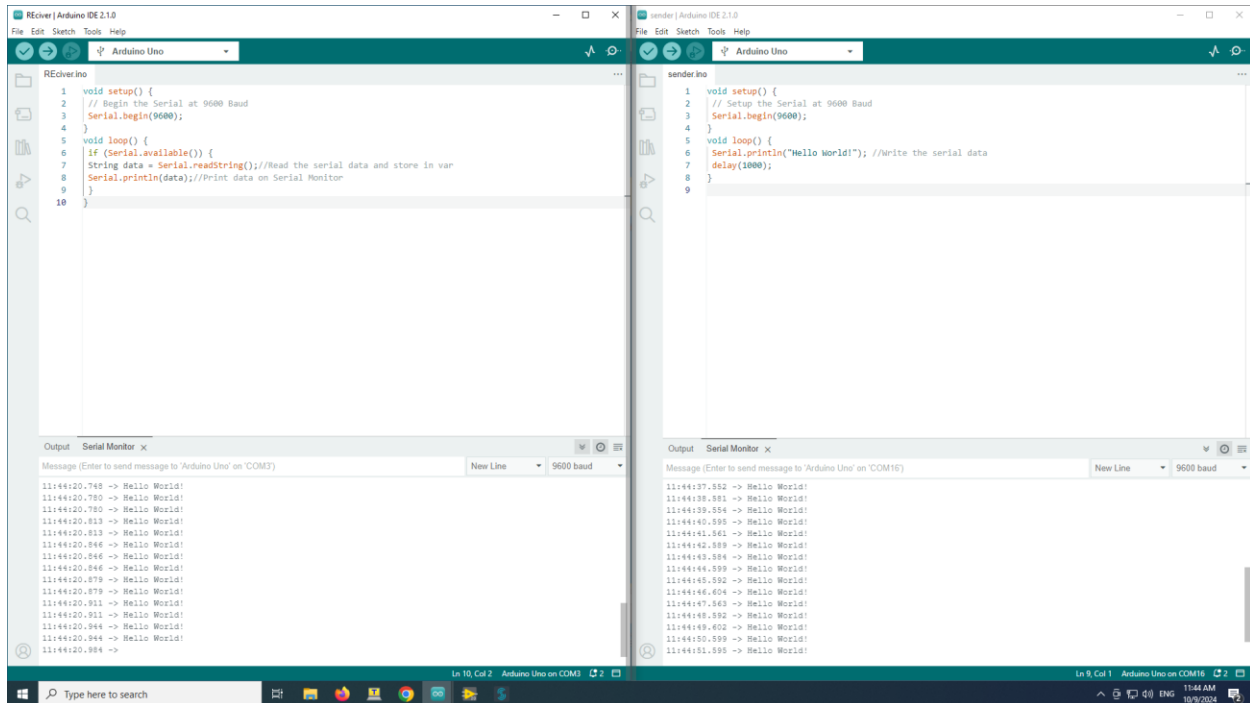


Figure 25: Part 2 Result

The sender code sends "Hello World!" each second across a serial connection. The receiver code receives the data that comes in, saves it as a string, and shows it in the Serial Monitor.

2.2.3 Push Button & LED using 2 Arduinos

At this part, a push button was attached to the Arduino1, and when pressed, it transferred the value 1 to Arduino2. Arduino2 was set up to get this value, and when it received a 1, an LED was switched on. Both Arduinos linked their TX and RX pins (TX1 ↔ RX2, RX1 ↔ TX2) and shared a common ground. The code was uploaded to both Arduinos, and its functionality was tested with the Serial Monitor.

NOTE: code in Appendix 2.3

Video of result: https://drive.google.com/file/d/1uhm5MNb3Aj1prlV2_15AtJOyspAK4IrY/view?usp=sharing

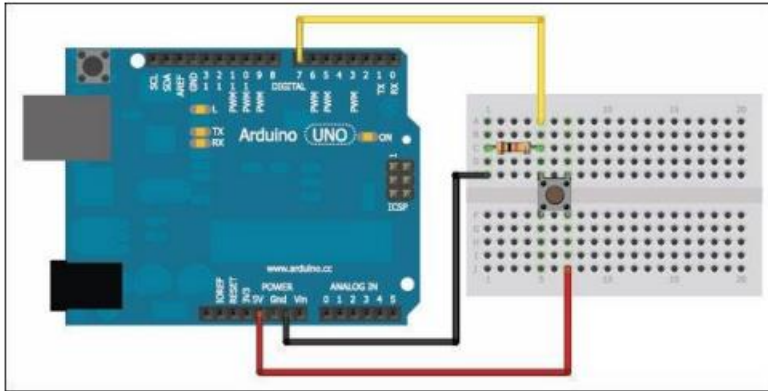


Figure 3: Arduino1 connected with Push button

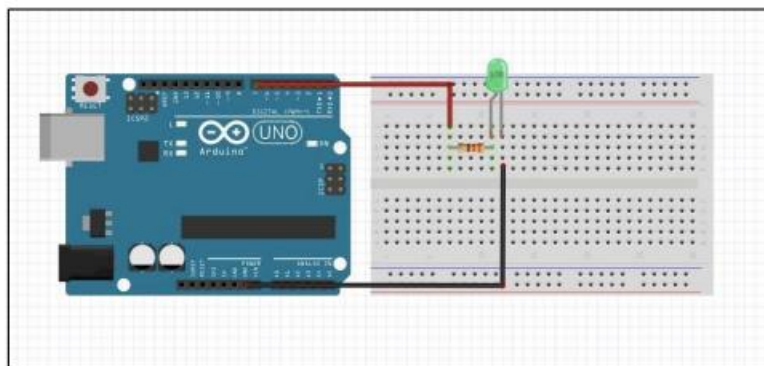


Figure 26: Part 3 circuit

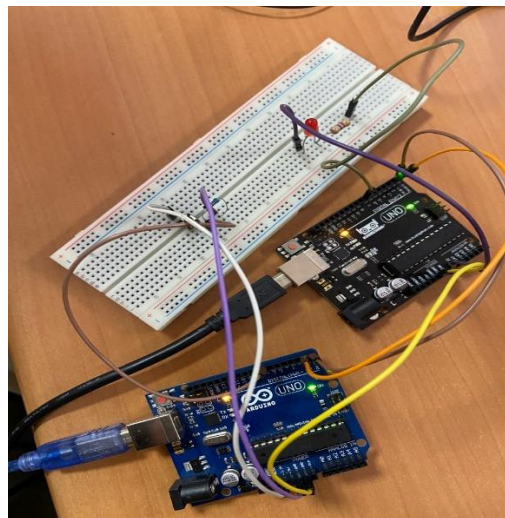


Figure 27: Part 3 connection

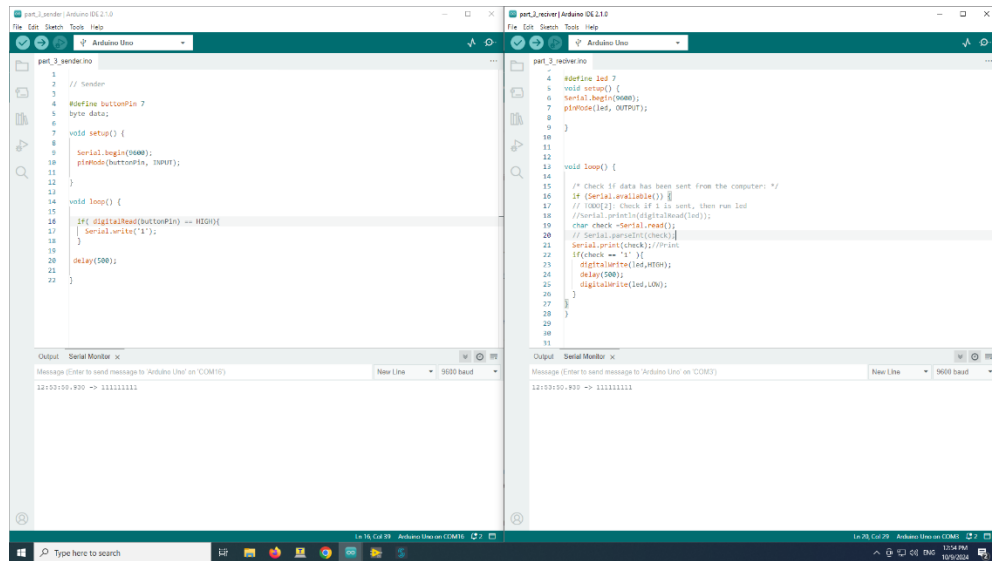


Figure 28: Part 3 Results

2.2.4 Visualization of serial communication using Serial Plotter

Throughout this section, the serial data transmission was illustrated by transferring a character from the sender Arduino to the receiver Arduino and plotting it as bits with the Serial Plotter. The sender's Baud rate was set low (300) to monitor the start, data, parity, and stop bits. To collect and show the bits, the receiver employed digitalRead and a high Baud rate (19200).

NOTE: code in Appendix 2.4

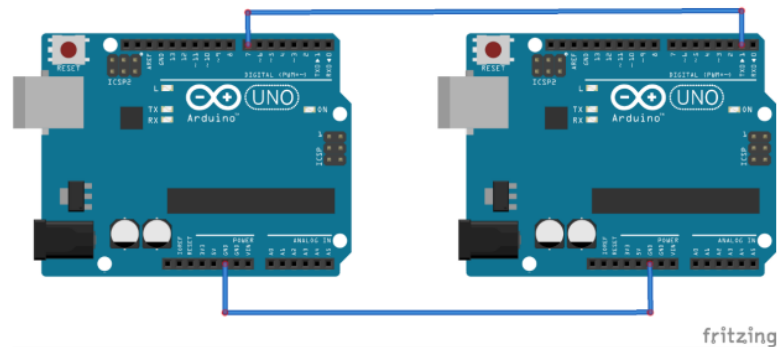


Figure 29: Part 4 Circuit

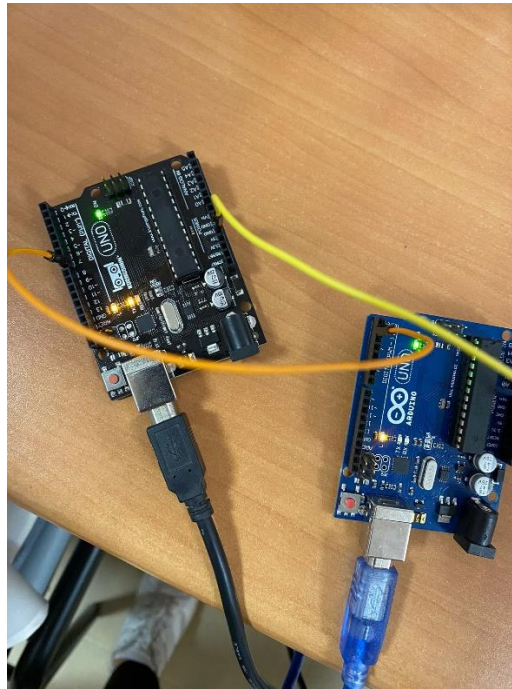


Figure 30: Part 4 Connection



Figure 31: Part 4 Result

2.2.5 TODO: LED Brightness control Via Push Buttons and Using UART

Two push buttons were utilized to adjust the intensity of an LED over the serial connection between two Arduinos. When Button 1 was hit, '1' was transmitted, raising the LED intensity, while Button 2 sent '2', reducing it. The brightness was altered in stages between 0 and 255, resulting in an LED that brightened or faded depending on which buttons were touched. The brightness was limited to 255 since it is the highest PWM value on an 8-bit pin and represents full brightness. Each button click increased the brightness by 25, but values remained between 0 and 255 for smooth LED control.

NOTE: code in Appendix 2.5

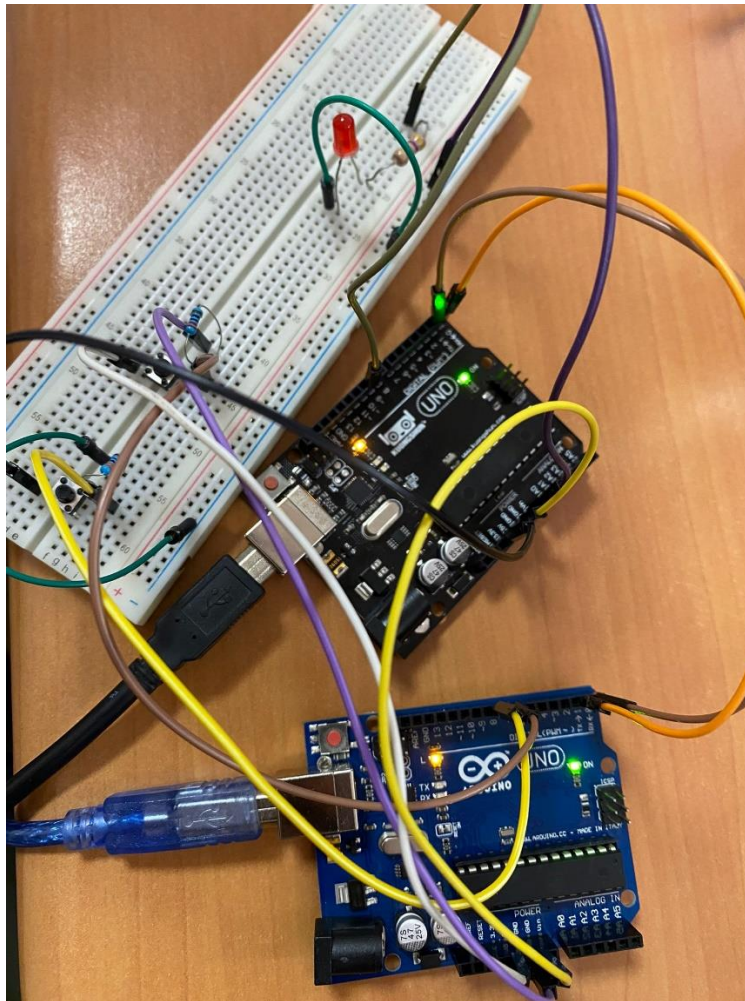


Figure 32:: Todo Connection

Video Of result:

https://drive.google.com/file/d/1BEw0jbRP2IQofSwm66bw3acKYTq_FzE3/view?usp=sharing

2.3 I2C in Arduino

2.3.1 Writing the LCD to Arduino and I2C scanning

The LCD was connected to the Arduino using the 5V, GND, SDA, and SCL pins as shown in the below figure.

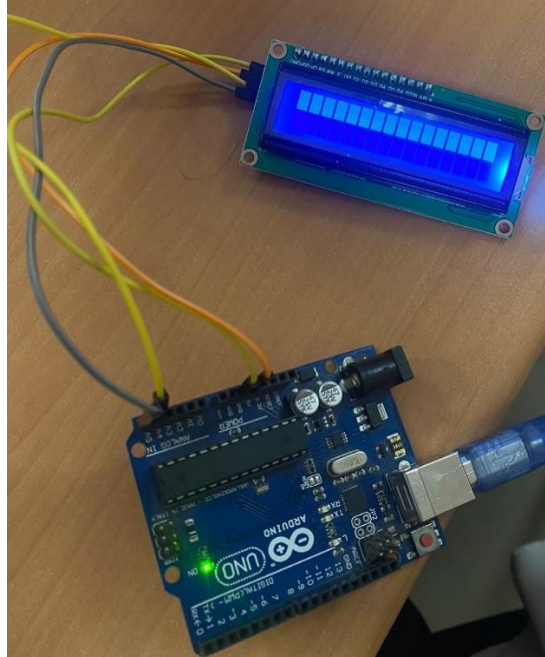


Figure 33: Find the address connection

After uploading the code provided in the appendix, the connection was successful, and the output was displayed on the serial monitor as follow:

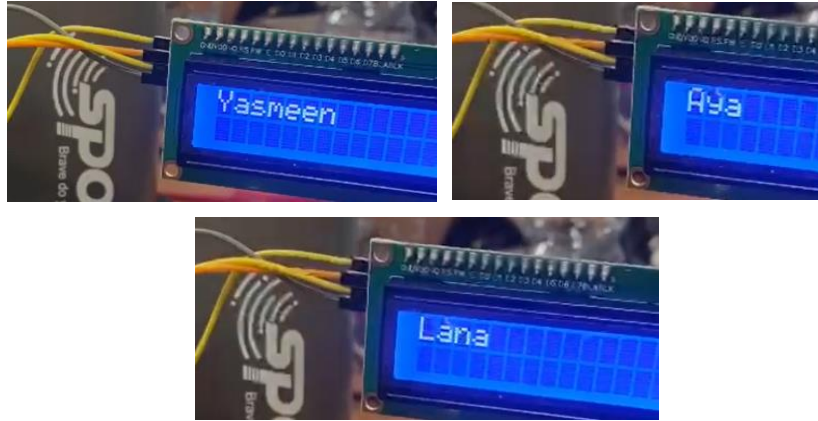
```
Output  Serial Monitor x
Message (Enter to send message to 'Arduino Uno' on 'COM13')
11:44:45.013 -> I2C device found at address 0x27
11:44:45.046 -> done
11:44:45.046 ->
```

Figure 34: Serial Monitor output for address

We will utilize address 0x27, which was the LCD's unique address while using the I2C, in the coming sections.

2.3.2: Display static text on the LCD

After configuring the LCD address to 0x27, the provided code was uploaded to the Arduino. The code was designed to display each name on the LCD, one at a time, with a delay between each change. This allowed for static text to be shown on the screen, transitioning between different names sequentially.



2.3.3: I2C communication between 2 Arduinos

The implementation of I2C communication was done between two Arduinos: one as a master and the other as a slave. We connected the devices according to the schematic shown in the figure below, making sure the GND is common.

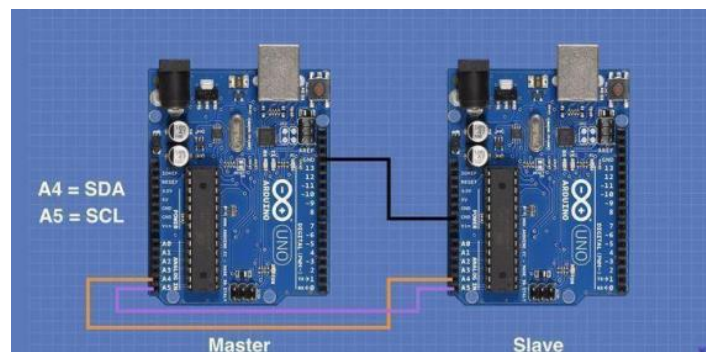


Figure 35: I2C communication between 2 Arduinos connection

Programmed the master to receive data from the slave; the slave transmitted sequentially character from '0' to 'z'. Having uploaded the respective master and slave codes in each Arduino, we started the serial monitor of the master unit and observed that the characters were received sequentially, thereby confirming us that the I2C communication is working as expected as follows:

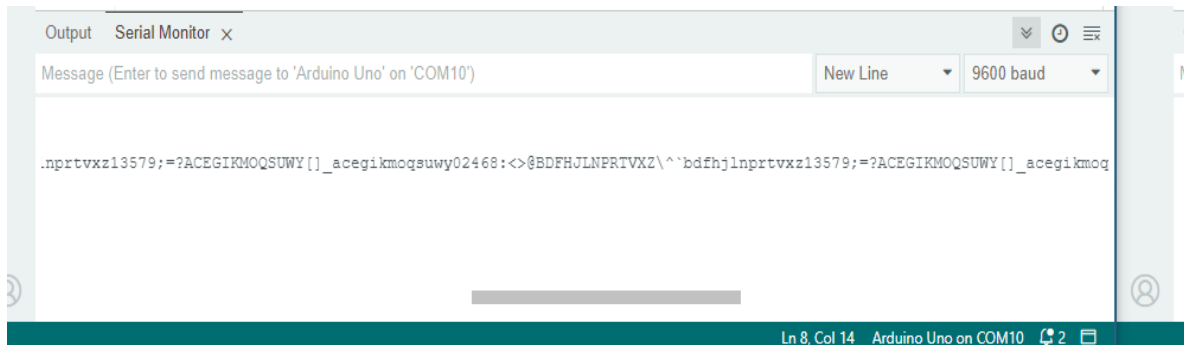


Figure 36: I2C communication between 2 Arduinos master receiving

2.3.4: I2C communication between 2 Arduinos and LCD

We provided communication between two Arduino devices using the I2C protocol and displayed analog readings from a potentiometer using an LCD connected to the master Arduino. From the previous section, we checked that the I2C connections were correctly set. An LCD was connected to the SDA and SCL pins from the master Arduino, while a potentiometer was attached to the A0 pin on the slave Arduino as shown in the figure:

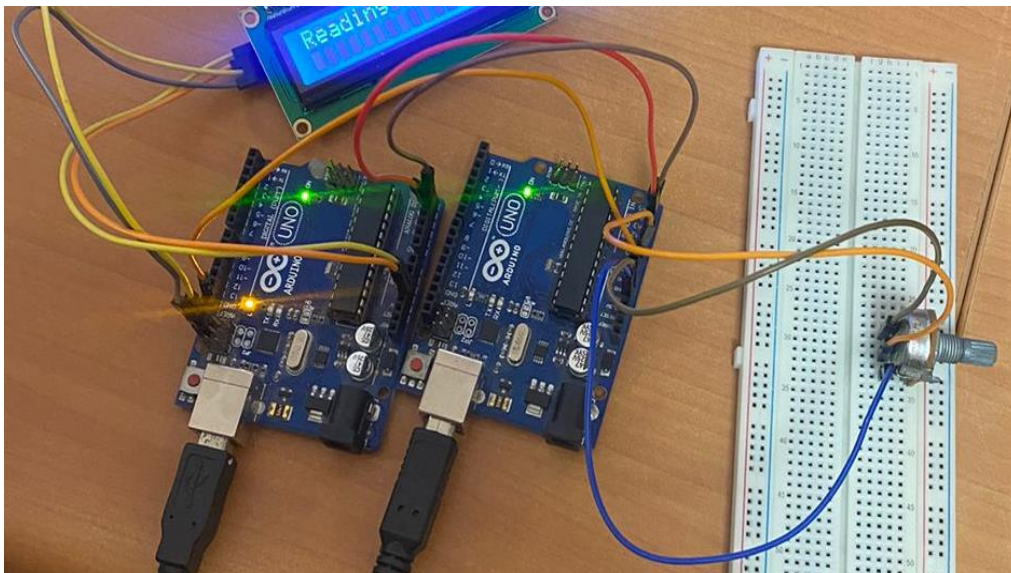


Figure 37: I2C communication between 2 Arduinos and LCD connection

After uploading the master and slave codes, the master read the analog values transmitted by the slave and displayed them on the LCD. The serial monitor gave confirmation of the data in transmission.



Figure 38: I2C communication between 2 Arduinos and LCD serial monitor & LCD output

2.3.5: I2C communication between Arduino and Temperature sensor (LM75B)

The LM75B temperature sensor was interfaced with the Arduino via I2C to measure the temperature, and the readings were displayed on the serial monitor, demonstrating the results as shown.



Figure 39: I2C communication between Arduino and Temperature sensor connection and serial monitor output

This Arduino code connects to an LM75B temperature sensor using the I2C protocol, assigning the sensor's address to 0x48.

2.3.6 ToDo

In this todo, an Arduino, an LCD, and a temperature sensor (LM75B) were used, the master writes on the LCD, while the slave reads from the sensor. The codes for the master and slave are included in the Appendix. The connection is shown in the figure below:

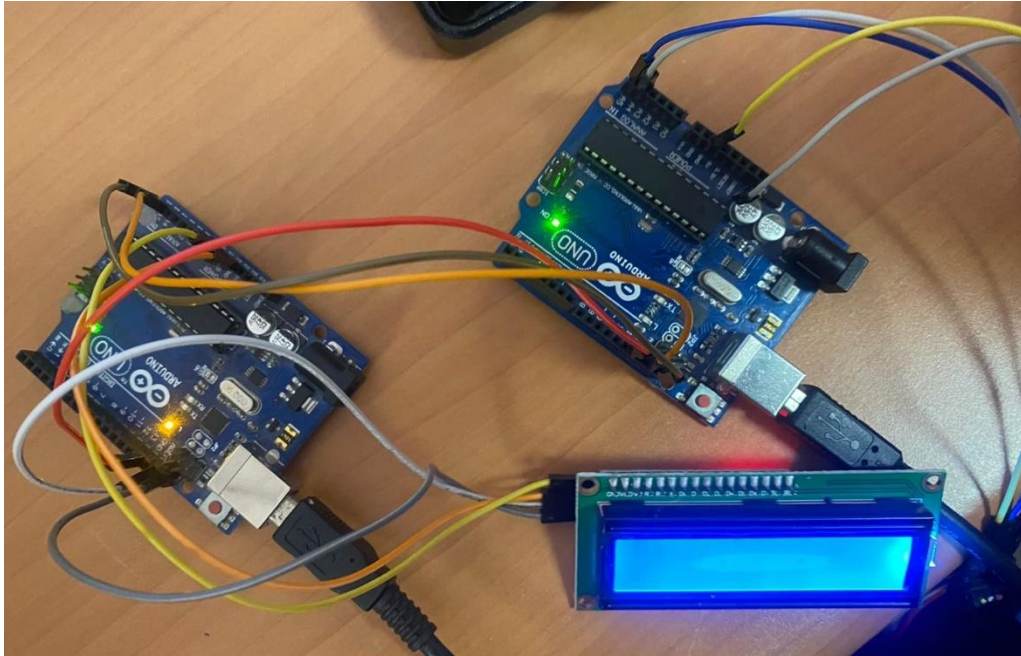


Figure 40: ToDo connection

Unfortunately, we were unable to fully execute this part because we waited for the temperature sensor to be available. After getting it, we didn't have enough time, like the rest of the students, to apply it correctly, but this is what we were able to achieve, and despite our understanding of how they operate

3. Conclusion

In conclusion, the experiments we conducted demonstrated the significant role Arduino plays in small-scale computing applications, particularly in enabling communication within embedded systems.

The first experiment explored the practical application of sensors with Arduino, highlighting the use of AnalogRead functions and the integration of both hardware and software interrupts. This basic understanding serves as a stepping stone for more complex projects.

In the second experiment, we focused on serial communication using the UART (Universal Asynchronous Receiver/Transmitter) protocol. We showed how Arduino facilitates direct device communication without the need for a clock signal, utilizing functions like Serial.read() and Serial.write(). This experiment demonstrated how effectively Arduino handles data transfer between boards, proving its capability in managing various communication scenarios.

The final experiment covered the I2C (Inter-Integrated Circuit) protocol, which allows multiple devices to communicate with each other. We discussed the roles of Master and Slave devices, illustrating how Arduino efficiently manages complex communication tasks. By integrating sensors and using an LCD display, we demonstrated how I2C makes device interactions smoother, further solidifying Arduino's reliability in creating responsive and efficient communication networks.

In summary, these experiments emphasized Arduino's essential role in serial communication and embedded systems, offering valuable insights into its practical applications. Whether for enthusiasts or professionals, these experiments provide the skills necessary for effective microcontroller-based communication.

Arduino is not just a tool—it's a platform for innovation, enabling the development of complex, interconnected systems that push the boundaries of technology. These experiments showcased how Arduino continues to drive advancements in the tech industry.

4. References

- [1] "arduino," [Online]. Available: <https://docs.arduino.cc/>. [Accessed 4 11 2024].
- [2] [Online]. Available: <https://www.rs-online.com/designspark/what-is-arduino-uno-a-getting-started-guide>. [Accessed 4 11 2024].
- [3] [Online]. Available: <https://components101.com/resistors/ldr-datasheet>. [Accessed 4 11 2024].
- [4] [Online]. Available: <https://www.sciencedirect.com/topics/engineering/photocell>. [Accessed 4 11 2024].
- [5] [Online]. Available: <https://learn.sparkfun.com/tutorials/processor-interrupts-with-arduino/all>. [Accessed 4 11 2024].
- [6] [Online]. Available: <https://deepbluembedded.com/arduino-software-interrupts/>. [Accessed 4 11 2024].
- [7] "Analog Dialogue," 19 10 2024. [Online]. Available: <https://www.analog.com/en/resources/analog-dialogue/articles/uart-a-hardware-communication-protocol.html>.
- [8] "Arduino Documents," Arduino, 17 10 2024. [Online]. Available: <https://docs.arduino.cc/learn/communication/uart/>. [Accessed 19 10 2024].
- [9] [Online]. Available: : <https://www.circuitbasics.com/basics-of-the-i2c-communication-protocol/> . [Accessed 1 11 2024].
- [10] [Online]. Available: <https://www.ti.com/lit/an/slva704/slva704.pdf> . [Accessed 1 11 2024].
- [11] [Online]. Available: <https://www.ocfreaks.com/i2c-tutorial/> . [Accessed 1 11 2024].
- [12] [Online]. Available: <https://www.design-reuse.com/articles/54776/i2c-interface-timing-specifications-and-constraints.html>. [Accessed 1 11 2024].

Appendix

Introduction to Arduino

Photoresistor

```
int photocellPin = 0; // the cell and 10K pulldown are connected to a0
int photocellReading; // the analog reading from the sensor divider
int LEDpin = 11; // connect Red LED to pin 11 (PWM pin)
int LEDbrightness; //
void setup(void) {
  // We'll send debugging information via the Serial monitor
  Serial.begin(9600);
}
void loop(void) {
  photocellReading = analogRead(photocellPin);
  Serial.print("Analog reading = ");
  Serial.print(photocellReading); // the raw analog reading

  if (photocellReading < 220) {
    Serial.println(" - Dark");
  }
  else if (photocellReading < 620) {
    Serial.println(" - Light");
  } else {
    Serial.println(" - bright");
  }

  // LED gets brighter the darker it is at the sensor
  // that means we have to -invert- the reading from 0-1023 back to 1023-0
  photocellReading = 1023 - photocellReading;
  //now we have to map 0-1023 to 0-255 since that's the range analogWrite uses
  LEDbrightness = map(photocellReading, 0, 1023, 0, 255);
  analogWrite(LEDpin, LEDbrightness);
  delay(1000);
}
```

Software Interrupts

```
// Timer1 overflow interrupt example

#define ledPin 13

void setup()
{
  pinMode(ledPin, OUTPUT);

  // initialize timer1
  noInterrupts(); // disable all interrupts
  TCCR1A = 0;
  TCCR1B = 0;
  TCNT1 = 34286; // preload timer 65536-16MHz/256/2Hz
  TCCR1B |= (1 << CS12); // 256 prescaler
  TIMSK1 |= (1 << TOIE1); // enable timer overflow interrupt
  interrupts(); // enable all interrupts
}

ISR(TIMER1_OVF_vect) // interrupt service routine that wraps a userdefined function supplied by
attachInterrupt
{
  TCNT1 = 34286; // preload timer
  digitalWrite(ledPin, digitalRead(ledPin) ^ 1);
}
```

Hardware Interrupts

```
// Hardware interrupt example

int pin = 13;
volatile int state = LOW;

void setup()
{
  pinMode(pin, OUTPUT);
  attachInterrupt(0, blink, CHANGE);
}

void loop()
{
  digitalWrite(pin, state);
}

void blink()
{
  state = !state;
}
```

UART

Appendix 2.1:

```
/* Use a variable called byteRead to temporarily store
the data coming from the computer */

byte byteRead;

void setup() {

// Turn the Serial Protocol ON

Serial.begin(9600);

}

void loop() {

/* check if data has been sent from the computer: */

if (Serial.available()) {

/* read the most recent byte */

byteRead = Serial.read();

/*ECHO the value that was read, back to the serial port. */

Serial.write(byteRead);

}}
```

Appendix 2.2:

```
// Receiver code

void setup() {

// Begin the Serial at 9600 Baud

Serial.begin(9600);

}

void loop() {

if (Serial.available()) {

String data = Serial.readString();//Read the serial data and store in var

Serial.println(data);//Print data on Serial Monitor

}

}

//-----

// Sender code
```



```

void setup() {
  // Setup the Serial at 9600 Baud
  Serial.begin(9600);
}

void loop() {
  Serial.println("Hello World!"); //Write the serial data
  delay(1000);
}

```

Appendix 2.3:

```

// Sender

#define buttonPin 7

byte data;

void setup() {
  Serial.begin(9600);
  pinMode(buttonPin, INPUT);
}

void loop() {
  if( digitalRead(buttonPin) == HIGH){
    Serial.write('1');
  }
  delay(500);
}

//-----

// Receiver

#define ledPin 13

byte data;

void setup() {
  Serial.begin(9600);
  pinMode(ledPin, OUTPUT);
}

void loop() {

```

```

if (Serial.available()) {

  data = Serial.read();

  if (data == '1') {

    digitalWrite(ledPin, HIGH); // Turn on LED

  }}

```

Appendix 2.4:

```

void setup() {

  // Setup the Serial at 300 Baud

  Serial.begin(300);

}

void loop() {

  Serial.write('A'); //Write the character A => will be transmitted as Byte [41H]

  delay(1500);

}

//-----

int readPin = 7;

void setup() {

  // Begin the Serial at 19200 Baud

  Serial.begin(19200);

  pinMode(readPin, INPUT);

}

void loop() {

  Serial.println(digitalRead(readPin));

}

```

Appendix 2.5:

```

#define led 9

int brightness = 0;

int step = 25;

void setup() {

```

```

Serial.begin(9600);

pinMode(led, OUTPUT);
}

void loop() {
  if (Serial.available()) {
    char check = Serial.read();
    Serial.print(check);

    if (check == '1') {
      brightness += step;
      if (brightness > 255) brightness = 255;
    }
    else if (check == '2') {
      brightness -= step;
      if (brightness < 0) brightness = 0;
    }

    analogWrite(led, brightness);
    delay(100);
  }
}

//-----

#define buttonPin1 7
#define buttonPin2 8

byte data;

void setup() {

```

```
Serial.begin(9600);

pinMode(buttonPin1, INPUT);
pinMode(buttonPin2, INPUT);
}

void loop() {
  if (digitalRead(buttonPin1) == HIGH) {
    Serial.write('1');
    delay(500);
  }

  if (digitalRead(buttonPin2) == HIGH) {
    Serial.write('2');
    delay(500);
  }
}
```

I2C in Arduino

Part 1:

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

void setup() {
    Wire.begin();
    Serial.begin(9600);
    Serial.println("\nI2C Scanner");
}

void loop() {
    byte error, address;
    int nDevices;
    Serial.println("Scanning...");
    nDevices = 0;
    for(address = 1; address < 127; address++ ) {
        Wire.beginTransmission(address);
        error = Wire.endTransmission();
        if (error == 0) {
            Serial.print("I2C device found at address 0x");
            if (address<16) {
                Serial.print("0");
            }
            Serial.println(address,HEX);
            nDevices++;
        }
        else if (error==4) {
            Serial.print("Unknow error at address 0x");
            if (address<16) {
                Serial.print("0");
            }
            Serial.println(address,HEX);
        }
    }
    if (nDevices == 0) {
        Serial.println("No I2C devices found\n");
    }
    else {
        Serial.println("done\n");
    }
    delay(5000);
}
```

Part 2

```
#include <LiquidCrystal_I2C.h>

// set the LCD number of columns and rows
int lcdColumns = 16;
int lcdRows = 2;
// set LCD address, number of columns and rows
// if you don't know your display address, run an I2C scanner sketch
```

```

LiquidCrystal_I2C lcd(0x27, lcdColumns, lcdRows);
void setup() {
  // initialize LCD
  lcd.init();
  // turn on LCD backlight
  lcd.backlight();
}
void loop() {
  // set cursor to first column, first row
  lcd.setCursor(0, 0);
  // print message
  lcd.print("Aya");
  delay(1000);
  // clears the display to print new message
  lcd.clear();

  lcd.print("Lana");
  delay(1000);
  lcd.clear();

  lcd.print("Yasmeen");
  delay(1000);
  lcd.clear();
}

```

Part 3

- Master

```

#include <Wire.h>
void setup()
{
  Wire.begin(); // join i2c bus (address optional for master)
  Serial.begin(9600); // start serial for output
  Serial.print("master sleeping...");
  delay(2000);
  Serial.println("go");
}
void loop()
{
  Wire.requestFrom(2, 1);
  while(Wire.available())
  {
    char c = Wire.read(); // receive a byte as character
    Serial.print(c); // print the character
  }
  delay(100);
}

```

- Slave

```

#include <Wire.h>
void setup()

```

```

{
Wire.begin(2);
Wire.onRequest(requestEvent); // register event
Serial.begin(9600); // start serial for output
}

void loop(){
delay(100);
}

void requestEvent(){
    static char c = '0';
    Wire.write(c++);
    if (c > 'z')
        c = '0';
}

```

Part 4

- Master

```

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
// set the LCD number of columns and rows
int lcdColumns = 16;
int lcdRows = 2;
// set LCD address, number of columns and rows
// if you don't know your display address, run an I2C scanner sketch
LiquidCrystal_I2C lcd(0x27, lcdColumns, lcdRows);
void setup()
{
Wire.begin();
// join i2c bus (address optional for master)
Serial.begin(9600); // start serial for output
Serial.print("master sleeping...");
delay(2000);
Serial.println("go");
// initialize LCD
lcd.init();
// turn on LCD backlight
lcd.backlight();
}

void loop()
{
Wire.requestFrom(50, 2);
// request data from slave device
int res;//result from I2c reading
byte MSB = Wire.read(); /* receive a byte MSB(NOTE: the function is
blocking, so it would not continue the code until a byte reach)*/
byte LSB = Wire.read(); /* receive a byte LSB(NOTE: the function is
blocking, so it would not continue the code until a byte reach)*/
res = ((MSB << 8) | LSB);
Serial.print("MSB = ");

```

```

Serial.print(MSB);
Serial.print(" , LSB = ");
Serial.print(LSB);
Serial.print(" , analog value = ");
Serial.println(res); // print the analog value
// set cursor to first column, first row
lcd.setCursor(0, 0);
// print message
lcd.print("Reading = ");
lcd.print(res);
    delay(1000);
    lcd.clear();
}

- Slave
#include <Wire.h>
void setup()
{
Wire.begin(50); // join i2c bus with address #2
Wire.onRequest(requestEvent); // register event
Serial.begin(9600); // start serial for output
}
void loop()
{
delay(100);
}
// function that executes whenever data is received from master
// this function is registered as an event, see setup()

void requestEvent()
{
int value = analogRead(A0);
Wire.write(value >> 8); // send the MSB
Wire.write(value & 0x00ff); // send the LSB
}

```

Part 5

```

#include <Wire.h>
#include <LiquidCrystal_I2C.h>

int LM75BAddress = 0x48;

void setup() {
    Serial.begin(9600);
    Wire.begin(); // Communication start
}

void loop() {
    float celsius = getTemperature();
    Serial.print("Celsius: ");
    Serial.println(celsius);
}

```



```

    float fahrenheit = (1.8 * celsius) + 32;
    Serial.print("Fahrenheit: ");
    Serial.println(fahrenheit);
    delay(200); // Just here to slow down the output. You can remove this
}

float getTemperature() {
    Wire.requestFrom(LM75BAddress, 2);
    byte MSB = Wire.read(); // Added '=' sign
    byte LSB = Wire.read(); // Added '=' sign
    // It's a 12-bit int, using two's compliment for negative
    int TemperatureSum = ((MSB << 8) | LSB) >> 4;
    float celsius = TemperatureSum * 0.0625;
    return celsius;
}

```

ToDo

- Master

```

#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// set the LCD number of columns and rows
int lcdColumns = 16;
int lcdRows = 2;

// set LCD address, number of columns and rows
LiquidCrystal_I2C lcd(0x27, lcdColumns, lcdRows);

void setup()
{
    Wire.begin(); // join i2c bus
    Serial.begin(9600); // start serial for output
    Serial.print("master sleeping...");
    delay(2000);
    Serial.println("go");
    // initialize LCD
    lcd.init();
    // turn on LCD backlight
    lcd.backlight();
}

void loop()
{
    Wire.requestFrom(50, 2); // request data from slave device
    delay(2000); // Adjust delay as needed
    int res;
}

void getTemperature() {
    Wire.requestFrom(LM75BAddress, 2);

```

```

    byte MSB = Wire.read();
    byte LSB = Wire.read();

    int res = ((MSB << 8) | LSB);
    lcd.setCursor(0, 0);
    // print
    lcd.print("Temp = ");
    lcd.print(res);
    delay(1000);
    lcd.clear();

}

- Slave
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

int LM75BAddress = 0x48;

void setup()
{
    Wire.begin(50);
    Wire.onRequest(requestEvent); // register event
    Serial.begin(9600); // start serial for output
}

void loop()
{
    getTemperature();
    delay(2000);
}

void requestEvent()
{
    int value = analogRead(A0);
    Wire.write(value >> 8); // send the MSB
    Wire.write(value & 0x00ff); // send the LSB
}

```